Signal Clone: PRD and Architecture

Secure Messaging Platform, SDE Fullstack Assignment

Version 1.0 | Stack: Next.js (TypeScript), FastAPI, SQLite, WebSockets

1. Overview

1.1 Purpose

Build a functional web clone of Signal Messenger that reproduces Signal's look, feel, and core messaging workflows. End-to-end encryption is simulated, not implemented.

1.2 Goals

- G1: All core features work correctly, including real-time messaging.
- G2: UI and UX are visually near-identical to Signal (Desktop and mobile).
- G3: Clean relational schema with proper relationships.
- G4: Clean, sensible API design and modular code.
- G5: Every line is explainable in the evaluation interview.

1.3 Non-goals

- Full Signal Protocol (X3DH, Double Ratchet) and multi-device key sync. A simplified Web Crypto E2EE for direct chats is in scope (FR-USP-1).
- Real SMS or phone verification.
- Voice and video calls, Stories, Linked devices (placeholders only).
- Native mobile apps.

1.4 Success criteria

- Two users in two browsers can chat live with correct ticks, typing indicators, and unread counts.
- Groups work with admin controls.
- App is seeded and usable immediately after deploy.
- Public GitHub repo, hosted demo link, and complete README delivered.

2. Users and Personas

Persona	Description	Key needs
Regular user	Registers with phone or username, chats 1:1	Fast messaging, receipts, search
Group member	Participates in group chats	See members, send and read messages
Group admin	Creates and manages a group	Add and remove members
Evaluator	Reviews the submission	Working demo, seed data, readable code, README

3. Scope and Priorities

- P0: Must have (assignment core). Required for submission.
- P1: High-value bonus. Build after P0 is stable.
- P2: Nice to have. Only with spare time.
- PH: Placeholder (Coming Soon screen only).

4. Functional Requirements

4.1 Authentication and Onboarding (FR-AUTH)

ID	Requirement	Priority
FR-AUTH-1	Register using a phone number or username	P0
FR-AUTH-2	Mock OTP verification with a fixed code (123456)	P0
FR-AUTH-3	Validate input (phone format, unique username, OTP length) with inline errors	P0
FR-AUTH-4	Profile setup: display name (required) and avatar (upload or generated initials)	P0
FR-AUTH-5	Login for existing users using phone or username plus mock OTP	P0
FR-AUTH-6	Session persistence across refresh using JWT (HTTP-only cookie or localStorage)	P0

ID	Requirement	Priority
FR-AUTH-7	Logout clears the session and disconnects the WebSocket	P0
FR-AUTH-8	Route guards: unauthenticated users are redirected to onboarding	P0
FR-AUTH-9	Edit profile (name, avatar, about) from settings	P1

4.2 Contacts and Conversation List (FR-CONV)

ID	Requirement	Priority
FR-CONV-1	Left pane lists all conversations (1:1 and group)	P0
FR-CONV-2	Sort by most recent activity (last message time), live-updated	P0
FR-CONV-3	Each row shows avatar, name, last-message preview, time, unread badge	P0
FR-CONV-4	Search conversations and contacts by name, username, or phone	P0
FR-CONV-5	Add a new contact by phone or username via modal, with not-found handling	P0
FR-CONV-6	Start a new chat from the compose button (contacts picker)	P0
FR-CONV-7	Unread count resets when the conversation is opened	P0
FR-CONV-8	Online indicator (green dot) driven by WebSocket presence	P0
FR-CONV-9	Last-seen text (for example, last seen today at 10:42), may be mocked	P0
FR-CONV-10	Typing preview in the list row (Typing…)	P2
FR-CONV-11	Filters: All, Unread, Groups	P1
FR-CONV-12	Archive and mute conversation (UI plus persisted flag)	P2

4.3 One-on-One Messaging (FR-DM)

ID	Requirement	Priority
FR-DM-1	Send and receive text messages in real time over WebSocket	P0
FR-DM-2	Messages persist in the database and load on reopen (paginated, newest first)	P0
FR-DM-3	Timestamps on messages, with day separators (Today, Yesterday, date)	P0
FR-DM-4	Message status: sending, sent, delivered, read	P0
FR-DM-5	Status icons: clock (sending), single check (sent), double check (delivered), filled or blue double check (read)	P0
FR-DM-6	Typing indicator shown to the other party, auto-expires after about 3 seconds	P0
FR-DM-7	Sent as delivered when the recipient has a live connection; read when the chat is open and visible	P0
FR-DM-8	Optimistic UI: message appears instantly as sending, reconciled by client_id	P0
FR-DM-9	Auto-scroll to latest, with a scroll-to-bottom button and new-message count	P0
FR-DM-10	Grouped bubbles (consecutive messages from the same sender are visually clustered)	P0
FR-DM-11	Empty message blocked; Enter sends, Shift+Enter inserts a newline	P0
FR-DM-12	Auto-reconnect WebSocket with backoff; offline messages queued and flushed	P1
FR-DM-13	Delete message (for me, for everyone)	P2

4.4 Group Messaging (FR-GRP)

ID	Requirement	Priority
FR-GRP-1	Create a group with a name, optional avatar, and 1 or more members	P0

ID	Requirement	Priority
FR-GRP-2	Creator becomes admin	P0
FR-GRP-3	Send and receive messages in a group in real time, fan-out to all members	P0
FR-GRP-4	Messages show the sender name and avatar in group bubbles	P0
FR-GRP-5	View group members with roles in a group info panel	P0
FR-GRP-6	Admin can add members	P0
FR-GRP-7	Admin can remove members; non-admins cannot (403)	P0
FR-GRP-8	System messages (X added Y, Y was removed, group created)	P0
FR-GRP-9	Members can leave the group	P1
FR-GRP-10	Admin can promote or demote other admins and rename the group	P1
FR-GRP-11	Group read receipts are tracked per member (delivered or read by N)	P1
FR-GRP-12	Group typing indicator (X is typing, X and Y are typing)	P1
FR-GRP-13	Removed users lose access to the group history from that point	P0

4.5 Signal Experience and UI (FR-UI)

ID	Requirement	Priority
FR-UI-1	Two-pane layout: conversation list on the left, chat pane on the right	P0
FR-UI-2	Signal-style bubbles: blue outgoing, grey incoming, rounded corners, tail-less clustering	P0
FR-UI-3	Chat header with avatar, name, presence or members, and an info button	P0

ID	Requirement	Priority
FR-UI-4	Composer with text input, emoji button, attach button, and send button	P0
FR-UI-5	Modals: new chat, add contact, create group, group info, profile	P0
FR-UI-6	Toasts and notifications for new messages, errors, and success actions	P0
FR-UI-7	Settings screen with sections: Profile, Privacy, Notifications, Appearance, Chats	P0
FR-UI-8	Settings are placeholders but toggles persist locally (and Appearance theme works)	P0
FR-UI-9	Placeholder Coming Soon screens: Voice and Video calls, Stories, Linked devices	PH
FR-UI-10	Mock encryption notice at the top of each chat (Messages are end-to-end encrypted)	P0
FR-UI-11	Safety number verification screen (see FR-USP-3)	P2
FR-UI-12	Loading skeletons, empty states, and error states throughout	P0
FR-UI-13	Browser tab title shows unread count; optional sound on new message	P2

4.6 Bonus Features (FR-BON)

ID	Requirement	Priority
FR-BON-1	Dark mode (follows system, manual toggle, persisted)	P1
FR-BON-2	Reply-to and quoted messages (quote block in bubble, click to jump to original)	P1
FR-BON-3	Emoji reactions (long-press or hover picker, per-user, live-synced)	P1
FR-BON-4	Responsive design: desktop (two panes), tablet, mobile (list or chat, back button)	P1

ID	Requirement	Priority
FR-BON-5	Keyboard shortcuts: Ctrl+K search, Esc close, Ctrl+N new chat, Up and Down to move through the list	P1
FR-BON-6	Attachments: images (preview, size limit 5 MB, stored on disk) and files	P2
FR-BON-7	Disappearing messages: per-chat timer (off, 30s, 5m, 1h, 1d), server-side expiry, clock icon	P2
FR-BON-8	Read-receipt privacy toggle that actually changes behavior	P2

4.7 USP Features (FR-USP)

These features stay inside Signal's visual language and strengthen its privacy story. Disappearing messages (FR-BON-7) is promoted from P2 to P1 and specified here. Build FR-USP-1 to FR-USP-4 first, as a block.

ID	Requirement	Priority
FR-USP-1	Real E2EE for direct chats with the Web Crypto API: an ECDH P-256 key pair per user generated in the browser, private key stored non-extractable in IndexedDB, AES-GCM key derived per conversation. The server stores only ciphertext and IV	P1
FR-USP-2	Public key registry: the client uploads its public key at registration and fetches the peer's key before the first message	P1
FR-USP-3	Safety number: a fingerprint derived from both public keys, shown as a number grid plus QR code in chat info, with Mark as verified showing a verified check	P1
FR-USP-4	Key-change warning: when a contact's active key changes, show a system message (Your safety number with X has changed) and reset the verified state	P1

ID	Requirement	Priority
FR-USP-5	Groups use transport security (TLS) only. The group chat shows a notice that groups are not end-to-end encrypted in this build, and the README documents the limitation	P1
FR-USP-6	Disappearing messages: per-chat timer (off, 30s, 5m, 1h, 1d), countdown ring on the bubble, server-side purge, system message when the timer changes. The timer applies only to messages sent after the change	P1
FR-USP-7	Note to Self: a conversation with yourself, created at registration and pinned at the top of the list	P1
FR-USP-8	Message info panel: per-recipient sent, delivered, and read timestamps, taken from message_receipts	P1
FR-USP-9	Block user: blocked users cannot message you or see your presence. Persisted and enforced server-side, with unblock in settings	P1
FR-USP-10	Working privacy toggles: hide message previews in the list and toasts, read receipts on or off, typing indicators on or off	P1
FR-USP-11	Screen lock: ask for the OTP again after inactivity (off, 1m, 5m, 30m)	P2

With E2EE on, the list preview and any in-chat search are decrypted client-side. Delivery and read receipts, unread counts, and ordering are unaffected because they use metadata only.

4.8 Security Requirements (FR-SEC)

ID	Requirement	Priority
FR-SEC-1	Object-level authorization on every REST endpoint and WebSocket event (no IDOR): a user cannot read another user's conversations, messages, receipts, or group members	P0

ID	Requirement	Priority
FR-SEC-2	Server-side validation: message body limit (4000 characters), upload size and MIME allowlist, Pydantic schemas on all inputs	P0
FR-SEC-3	Rate limiting on OTP requests and verification (5 per 10 minutes per phone and IP), returning 429 with Retry-After	P1
FR-SEC-4	WebSocket hardening: token auth, origin check, per-connection message rate limit	P1
FR-SEC-5	Security headers (CSP, X-Content-Type-Options, Referrer-Policy) and a CORS allowlist	P1
FR-SEC-6	JWT with short expiry (for example 24 hours); secrets loaded from environment variables only	P1
FR-SEC-7	SECURITY.md describing the threat model, what is and is not encrypted, and known limitations	P1
FR-SEC-8	Automated tests covering authorization (cross-user access returns 403 or 404) and rate limiting	P1

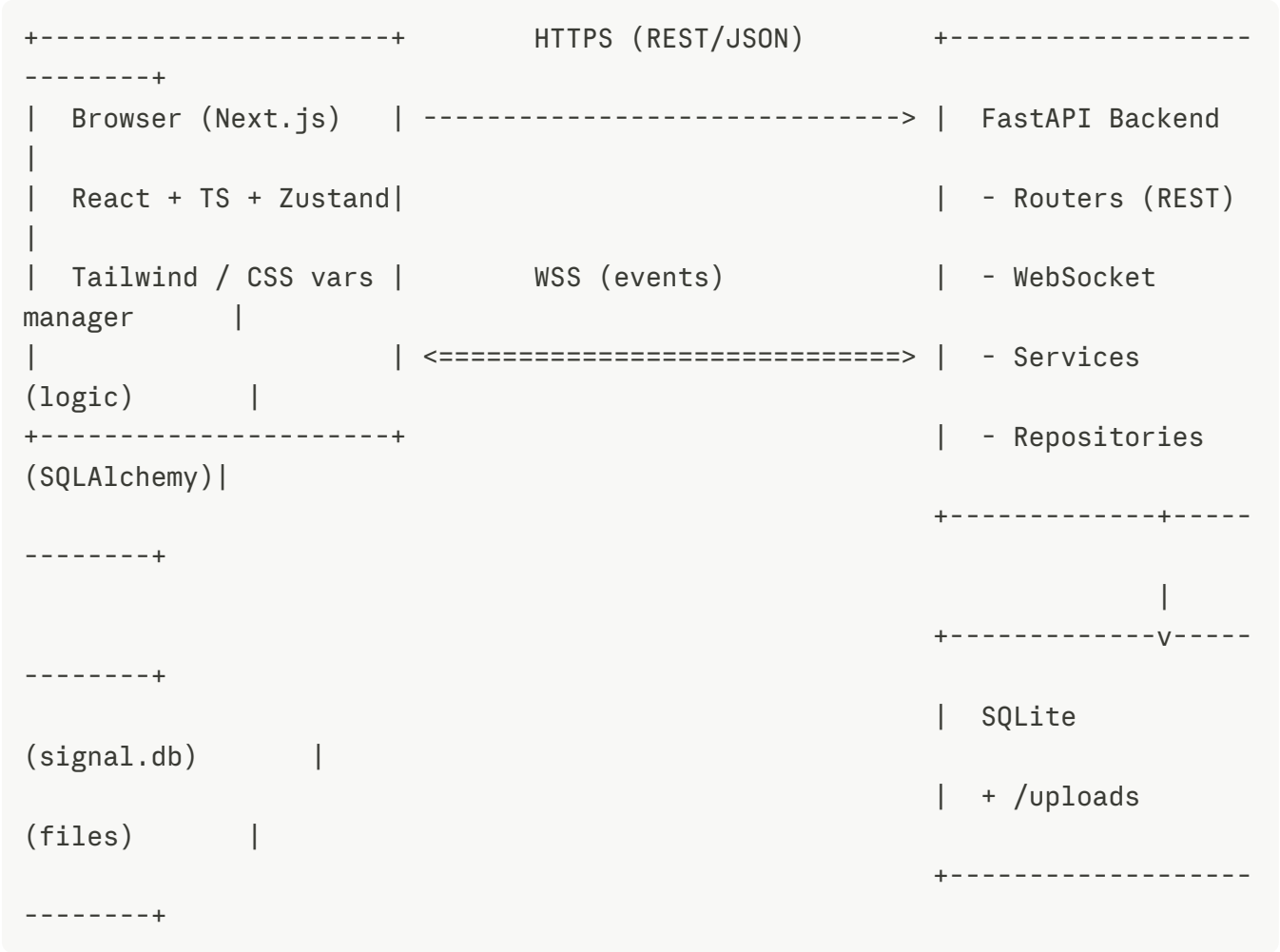
5. Non-Functional Requirements

ID	Category	Requirement
NFR-1	Performance	Message round-trip under 300 ms locally; list loads under 1 s with seed data
NFR-2	Scalability	Cursor-based pagination for messages (50 per page); indexes on hot columns
NFR-3	Reliability	WebSocket auto-reconnect; idempotent sends via client_id; no duplicate messages
NFR-4	Security	JWT auth on REST and WebSocket; input validation; membership checks on every conversation action; CORS allowlist
NFR-5	Privacy	E2EE for direct chats via Web Crypto (FR-USP-1); groups use TLS only and this is documented. Do not claim Signal Protocol. Passwords are not used (OTP only)

ID	Category	Requirement
NFR-6	Usability	Keyboard accessible, focus states, ARIA labels, contrast in both themes
NFR-7	Compatibility	Latest Chrome, Firefox, Safari, Edge; viewports from 360 px up
NFR-8	Maintainability	Layered backend, feature-based frontend, typed (TypeScript and Pydantic), linted
NFR-9	Observability	Structured logs; /health endpoint
NFR-10	Portability	One-command local setup; seed script is idempotent and runs on startup if the DB is empty

6. System Architecture

6.1 High-level diagram



6.2 Deployment topology

```
Vercel (frontend)  --REST/WSS-->  Render or Railway (FastAPI + SQLite + uploads volume)
```

Vercel cannot hold long-lived WebSocket servers, so the backend is hosted separately. Use a persistent disk or run the seed on startup, because free-tier disks can reset on redeploy.

6.3 Architectural principles

- Layered backend: router, service, repository, model. Routers hold no business logic.
- REST for state and history, WebSocket for live events only.
- The server is the source of truth for message status and ordering.
- Single WebSocket connection per client, multiplexed by event type.
- Frontend organized by feature with shared UI primitives.

7. Technology Stack

Layer	Choice	Reason
Frontend	Next.js 14+ (App Router), TypeScript	Required by the assignment
Styling	Tailwind CSS plus CSS variables for theming	Fast pixel-matching, easy dark mode
State	Zustand (or Redux Toolkit)	Light global state for chats, presence, auth
Data fetching	TanStack Query or fetch wrappers	Caching, pagination
Backend	FastAPI (Python 3.11+)	Native WebSocket support, async, auto OpenAPI docs
ORM	SQLAlchemy 2.x plus Alembic	Relationships and migrations
Validation	Pydantic v2	Typed request and response schemas
Database	SQLite	Required by the assignment
Auth	JWT (python-jose or PyJWT)	Stateless sessions
Real-time	FastAPI WebSockets	No extra infrastructure

Layer	Choice	Reason
Testing	Pytest, httpx, React Testing Library (optional)	Core API coverage
Hosting	Vercel plus Render or Railway	Free tiers

8. Backend Architecture

8.1 Folder structure

```
backend/
  app/
    main.py              # app factory, CORS, router registration
    core/                # config, security (JWT), dependencies
    db/                  # session, base, seed.py
    models/              # SQLAlchemy models
    schemas/             # Pydantic schemas
    repositories/        # DB access per entity
    services/            # business logic (auth, chat, group, receipts)
    api/                 # REST routers: auth, users, contacts,
conversations, messages, groups, uploads
    ws/                  # connection manager, event handlers, presence
  tests/
  requirements.txt
  .env.example
```

8.2 Key components

- **Connection manager:** maps user_id to a set of active sockets; supports send to user and broadcast to a conversation.
- **Presence service:** marks a user online on connect and offline on the last disconnect; updates last_seen.
- **Message service:** validates membership, persists the message, creates receipt rows, fans out events.
- **Receipt service:** updates delivered and read per recipient and derives the aggregate status.
- **Expiry worker (P2):** background task that deletes expired disappearing messages and emits events.

9. Frontend Architecture

9.1 Folder structure

```
frontend/
  src/
    app/                 # routes: /welcome, /verify, /profile,
/(main)/chats/[id], /settings
    features/
      auth/              # forms, hooks
      conversations/     # list, search, new chat, filters
      chat/              # message list, bubble, composer, status ticks,
typing
      groups/            # create group, info panel, member management
      settings/          # settings screens, theme
    components/ui/       # Avatar, Modal, Toast, Button, Input, Badge,
IconButton
    lib/                 # api client, ws client, utils, date formatters
    store/               # auth, conversations, messages, presence, ui stores
    styles/              # tokens, globals
    types/
```

9.2 State and data flow

- REST loads initial lists and history; the WebSocket client pushes events into the stores.
- Optimistic send: insert with a temporary client_id, status sending, then replace on message.ack.
- A single WebSocket provider mounted in the main layout handles reconnect with exponential backoff.
- Read marking uses IntersectionObserver or chat-focus to emit message.read.

10. Database Design

10.1 Entity relationships

```
users 1---* contacts (owner) ; users 1---* contacts (contact)
users *---* conversations   (through conversation_members)
conversations 1---* messages
users 1---* messages (sender)
messages 1---* message_receipts (per recipient)
messages 1---* reactions
messages 1---* attachments
messages 0..1---1 messages (reply_to)
```

10.2 Tables

users

Column	Type	Notes
id	INTEGER PK	
phone	TEXT UNIQUE NULL	E.164 format
username	TEXT UNIQUE NULL	At least one of phone or username required
display_name	TEXT NOT NULL	
about	TEXT	
avatar_url	TEXT	
last_seen_at	DATETIME	
is_online	BOOLEAN	Derived from presence, cached
settings_json	TEXT	Privacy and notification toggles
created_at	DATETIME	

contacts

Column	Type	Notes
id	INTEGER PK	
owner_id	FK users	
contact_user_id	FK users	

Column	Type	Notes
nickname	TEXT	Optional
created_at	DATETIME	

Unique (owner_id, contact_user_id).

conversations

Column	Type	Notes
id	INTEGER PK	
type	TEXT	direct or group
name	TEXT NULL	Group only
avatar_url	TEXT NULL	Group only
created_by	FK users	
direct_key	TEXT UNIQUE NULL	Sorted user ids, prevents duplicate 1:1 chats
disappearing_seconds	INTEGER NULL	P2
last_message_id	FK messages NULL	Denormalized for fast list sorting
last_activity_at	DATETIME	Indexed
created_at	DATETIME	

conversation_members

Column	Type	Notes
id	INTEGER PK	
conversation_id	FK	
user_id	FK	
role	TEXT	admin or member
joined_at	DATETIME	
left_at	DATETIME NULL	Set on removal or leave
last_read_message_id	FK messages NULL	Drives unread counts
is_muted / is_archived	BOOLEAN	P2

Unique (conversation_id, user_id).

messages

Column	Type	Notes
id	INTEGER PK	
conversation_id	FK	Indexed with created_at
sender_id	FK users NULL	NULL for system messages
client_id	TEXT	Idempotency key, unique per sender
type	TEXT	text, system, image, file
body	TEXT	
reply_to_id	FK messages NULL	P1
status	TEXT	sent, delivered, read (aggregate)
expires_at	DATETIME NULL	P2
deleted_at	DATETIME NULL	Soft delete
created_at	DATETIME	

message_receipts

Column	Type	Notes
id	INTEGER PK	
message_id	FK	
user_id	FK	Recipient
delivered_at	DATETIME NULL	
read_at	DATETIME NULL	

Unique (message_id, user_id). For direct chats there is one row; for groups, one per member. The message aggregate status is derived from these rows.

reactions

Column	Type	Notes
id	INTEGER PK	
message_id	FK	
user_id	FK	
emoji	TEXT	
created_at	DATETIME	

Unique (message_id, user_id): one reaction per user per message, replaced on change.

attachments (P2)

Column	Type	Notes
id	INTEGER PK	
message_id	FK	
file_url	TEXT	
mime_type	TEXT	
size_bytes	INTEGER	
width / height	INTEGER	Images

10.3 Indexes

- messages (conversation_id, created_at DESC)
- conversation_members (user_id) and (conversation_id)
- conversations (last_activity_at DESC)
- message_receipts (user_id, read_at)
- users (phone), users (username)

10.4 Unread count rule

Unread = messages in the conversation with id greater than the member's last_read_message_id, sent by someone else, and created after joined_at.

10.5 Schema Additions for USP Features

New table	Columns	Purpose
user_keys	id, user_id (FK), public_key_jwk, fingerprint, created_at, revoked_at	Public key registry. The newest non-revoked row is the active key; unique (user_id, fingerprint)
blocks	id, blocker_id (FK), blocked_id (FK), created_at	Block list; unique (blocker_id, blocked_id)
contact_verifications	id, owner_id (FK), contact_user_id (FK), verified_fingerprint, verified_at	Safety number verified state per contact

Changes to existing tables:

- messages: add ciphertext (TEXT NULL), iv (TEXT NULL), sender_key_id (FK user_keys NULL), is_encrypted (BOOLEAN, default false). body is NULL when is_encrypted is true.
- conversations: type gains the value note_to_self.
- users.settings_json: keys hide_previews, read_receipts, typing_indicators, screen_lock_minutes.
- message_receipts: unchanged. It also powers the message info panel.

11. REST API Overview

Base path: /api/v1. All endpoints except auth require a Bearer token.

11.1 Auth and users

Method	Endpoint	Purpose
POST	/auth/request-otp	Start registration or login (mock, returns success)
POST	/auth/verify-otp	Verify 123456, return JWT and is_new_user flag
POST	/auth/profile	Set display name and avatar (onboarding)
POST	/auth/logout	Invalidate the session client-side and mark offline
GET	/users/me	Current user
PATCH	/users/me	Update profile and settings
GET	/users/search?q=	Search users by name, phone, or username

11.2 Contacts

Method	Endpoint	Purpose
GET	/contacts	List contacts
POST	/contacts	Add a contact by phone or username
DELETE	/contacts/{id}	Remove a contact

11.3 Conversations and messages

Method	Endpoint	Purpose
GET	/conversations?q=&filter=	List with last message, unread count, presence; searchable
POST	/conversations	Get or create a direct conversation with a user
GET	/conversations/{id}	Conversation details
GET	/conversations/{id}/messages?before=&limit=	Paginated history
POST	/conversations/{id}/messages	REST fallback send (client_id for idempotency)
POST	/conversations/{id}/read	Mark read up to a message id

Method	Endpoint	Purpose
PATCH	/conversations/{id}/settings	Mute, archive, disappearing timer
DELETE	/messages/{id}	Delete message (P2)
PUT / DELETE	/messages/{id}/reaction	Set or remove a reaction (P1)
POST	/uploads	Upload an attachment (P2)

11.4 Groups

Method	Endpoint	Purpose
POST	/groups	Create group (name, avatar, member_ids)
GET	/groups/{id}/members	List members with roles
POST	/groups/{id}/members	Add members (admin only)
DELETE	/groups/{id}/members/{user_id}	Remove member (admin only) or leave (self)
PATCH	/groups/{id}	Rename, change avatar (admin only)
PATCH	/groups/{id}/members/{user_id}/role	Promote or demote (admin only)

11.5 USP Endpoints

Method	Endpoint	Purpose
PUT	/keys/me	Register or rotate the caller's public key
GET	/keys/{user_id}	Fetch a user's active public key and fingerprint
POST / DELETE	/contacts/{id}/verify	Mark or clear safety number verification
GET	/messages/{id}/info	Per-recipient receipt timestamps (sender only)
GET / POST	/blocks	List blocked users, block a user
DELETE	/blocks/{user_id}	Unblock

Method	Endpoint	Purpose
POST	/conversations/note-to-self	Get or create the Note to Self conversation

11.6 Conventions

- JSON responses; ISO-8601 UTC timestamps.
- Error shape: { error: { code, message } }.
- Status codes: 200, 201, 400, 401, 403, 404, 409, 422.
- Auto-generated OpenAPI docs at /docs.

12. WebSocket Protocol

Endpoint: /ws?token=JWT. Messages are JSON: { type, data }.

12.1 Client to server

Event	Payload	Purpose
message.send	conversation_id, client_id, body, reply_to_id?	Send a message
typing.start / typing.stop	conversation_id	Typing indicator
message.delivered	message_id	Ack on receipt (auto-sent by client)
message.read	conversation_id, up_to_message_id	Mark read
reaction.set	message_id, emoji	React (P1)
ping		Keepalive

12.2 Server to client

Event	Payload	Purpose
message.ack	client_id, message_id, created_at, status=sent	Confirms persistence
message.new	full message object	New message for recipients

Event	Payload	Purpose
message.status	message_id, status, user_id?	Delivered or read updates
typing	conversation_id, user_id, is_typing	Typing indicator
presence	user_id, is_online, last_seen_at	Online and last-seen
conversation.updated	conversation summary	List refresh (new group, rename, member change)
member.added / member.removed	conversation_id, user_id	Group changes
reaction.updated	message_id, reactions	Live reactions
message.deleted / message.expired	message_id	Removal
error	code, message	Failures

12.3 Message lifecycle

```
Client: render bubble (sending, client_id)
  -> message.send
Server: validate membership, persist, create receipts
  -> message.ack to sender            => status: sent (single check)
  -> message.new to online recipients
Recipient client: message.delivered
  -> server updates receipt, emits message.status => delivered (double check)
Recipient opens chat: message.read
  -> server updates receipt, emits message.status => read (blue double check)
```

Offline recipients receive undelivered messages on reconnect: the server sends a sync of pending messages and marks them delivered once acked.

12.4 Group status aggregation

For groups, status is sent when persisted, delivered when all active members have delivered, and read when all active members have read. Per-member detail is available from message_receipts.

12.5 Additional Events for USP Features

Event	Direction	Notes
message.send (encrypted)	Client to server	Carries ciphertext, iv, and sender_key_id instead of body for direct chats
key.changed	Server to client	user_id and new fingerprint; triggers the key-change system message
error (blocked)	Server to client	Returned when the recipient has blocked the sender; no delivery or typing events are sent
message.expired	Server to client	Emitted when the purge job deletes a disappearing message

13. Authorization Rules

- A user can only read or send in conversations where they are an active member (left_at is NULL).
- Only admins can add, remove, rename, or change roles.
- Only the sender can delete their own message (for everyone).
- Direct conversations are unique per user pair (direct_key).
- WebSocket events are checked against membership on every message.

14. UI and UX Specification

14.1 Design tokens (verify against real Signal while building)

Token	Light	Dark
Primary (outgoing bubble, accents)	#3A76F0	#3A76F0
Incoming bubble	#E9E9E9	#2B2B2B
Background	#FFFFFF	#121212
Sidebar background	#F6F6F6	#1B1B1B
Text primary	#1B1B1B	#E9E9E9
Text secondary	#6E6E6E	#A0A0A0
Unread badge	#3A76F0	#3A76F0

Token	Light	Dark
Online dot	#2CB67D	#2CB67D

Typography: system sans-serif stack (Inter as the closest fallback). Bubble radius about 18 px, list row height about 72 px, avatar 48 px in list and 36 px in chat.

14.2 Screens

1. Welcome and phone or username entry
2. OTP verification
3. Profile setup (name and avatar)
4. Main view: conversation list and chat pane (empty state when none selected)
5. New chat and add contact modals
6. Create group flow (select members, then name)
7. Group info panel (members, roles, add, remove, leave)
8. Settings (Profile, Privacy, Notifications, Appearance, Chats, Linked devices placeholder)
9. Coming Soon screens (Calls, Stories, Linked devices)

14.3 Interaction details

- Bubble clustering with timestamp and tick on the last bubble of a cluster.
- Hover actions on a bubble: react, reply, more.
- Typing bubble with animated dots.
- Toast on new message when the chat is not focused.
- Mobile: list and chat as separate views with a back arrow.

15. Seed Data

Seed script (idempotent, runs when the DB is empty):

- 8 to 10 users with names, avatars, and mixed online and last-seen states.
- One demo account with a known phone and OTP (document in README).
- Contacts for the demo user.
- 5 or more direct conversations with 15 to 40 messages each across several days, mixed statuses.
- 2 to 3 groups with mixed roles, system messages, and group chatter.
- Some unread messages, a few replies and reactions (if built).

- Provide a second demo account so evaluators can test real-time in two windows.

16. Testing Strategy

Level	Scope
Backend unit	Services: receipts aggregation, unread count, membership checks
API tests	Auth, send message, pagination, group admin permissions (403 cases)
WebSocket tests	Send, ack, delivered, read, typing flow
Manual	Two-browser real-time scenarios, reconnect, mobile viewport
UI	Visual comparison against Signal screenshots

17. Deployment and DevOps

- Frontend: Vercel, env NEXT_PUBLIC_API_URL and NEXT_PUBLIC_WS_URL.
- Backend: Render or Railway, start command uvicorn app.main:app --host 0.0.0.0 --port $PORT.
- Run migrations and seed on startup. Mount a persistent volume for SQLite and uploads if available.
- Configure CORS for the Vercel domain; use wss:// in production.
- Keep .env.example files in both folders; never commit secrets.
- Free-tier backends sleep when idle; mention the cold start in the README.

18. Milestones (about 24 hours)

Phase	Hours	Deliverables
1. Foundation	2	Repos, project setup, schema, migrations, seed skeleton
2. Auth	2	OTP flow, JWT, profile setup, route guards
3. Messaging core	5	REST plus WebSocket, persistence, receipts, typing, presence

Phase	Hours	Deliverables
4. Conversation list	2	List, search, unread, add contact
5. Groups	3	Create, members, admin controls, system messages
6. UI pixel pass	5	Signal styling, modals, toasts, settings, placeholders
7. Bonus features	3	Dark mode, replies, reactions, responsive, shortcuts
8. Ship	2	README, ER diagram, seed polish, deploy, final QA

If time is short, cut P2 first, then P1 bonus items from the bottom up. Never cut seed data, README, or deployment.

USP time budget: E2EE with safety number and key-change warning takes about 3 hours, security hardening with tests about 2 hours, and Note to Self, message info, block, and privacy toggles about 2 hours. That is about 7 extra hours, so protect them by dropping lower-value bonus items first (attachments, then keyboard shortcuts). Keep dark mode, replies, and reactions.

19. Risks and Mitigations

Risk	Mitigation
WebSocket hosting limits	Split deploy: Vercel plus Render or Railway
SQLite data reset on redeploy	Idempotent seed on startup; persistent disk
Pixel-matching takes longer than planned	Build design tokens and UI primitives first; reuse everywhere
Race conditions or duplicate messages	client_id idempotency, server-assigned ordering
Group receipt complexity	Per-recipient receipts table from day one
Scope creep	Strict P0 then P1 then P2 order
Cannot explain code in interview	Keep layers simple; review every AI-generated file

20. Deliverables Checklist

- [ ] Public GitHub repository with frontend/ and backend/
- [ ] Hosted frontend and backend, working demo link
- [ ] README: setup, tech stack, architecture overview, DB schema (with ER diagram), API overview, assumptions, demo credentials
- [ ] Seeded database, usable immediately
- [ ] All P0 requirements implemented and tested
- [ ] At least dark mode, replies, and reactions from the bonus list
- [ ] Original work, with no copied repositories
- [ ] SECURITY.md with threat model and known limitations
- [ ] README section explaining the E2EE design, key storage, and the group-chat limitation
- [ ] Authorization and rate-limit tests passing
- [ ] At least FR-USP-1 to FR-USP-4, FR-USP-6, and FR-SEC-1 to FR-SEC-3 implemented

21. Acceptance Criteria (summary)

1. A new user registers with OTP 123456, sets a name and avatar, and lands on the main view.
2. Two users chat live; the sender sees the clock, then single check, then double check, then read state as the recipient acts.
3. Typing indicator appears and disappears correctly.
4. Unread badges and previews update live, and the list re-sorts by recent activity.
5. Messages persist after refresh and after logout and login.
6. A group is created; all members receive messages; an admin can add and remove members; a non-admin gets 403.
7. Online and last-seen indicators update on connect and disconnect.
8. Settings screens exist, and Coming Soon screens show for calls, stories, and linked devices.
9. The UI is visually consistent with Signal in light and dark mode on desktop and mobile widths.

22. Assumptions

- Fixed OTP 123456 for all users.

- Direct chats use a simplified Web Crypto E2EE scheme; group messages and seeded messages are stored in plaintext (seed rows are flagged is_encrypted = false). Private keys live in the browser, so logging in from a new browser creates a new key and triggers the key-change warning.
- Phone numbers are not verified beyond format.
- Single deployment region; no horizontal scaling (single WebSocket process).
- Uploaded files are stored on local disk with a size limit.